

OWASP A10:2025 Mishandling of Exceptional Condition

Di Dio Giuseppe Loris, Matricola 1000032276

May 21, 2026



Università di Catania

Dipartimento di Matematica e Informatica

Docenti: Giampaolo Bella e Sergio Esposito

Indice

1	Descrizione	3
1.1	Perché Mishandling of Exceptional Condition è Critica	3
1.1.1	Il Principio del "Fallimento Sicuro" (Fail-Safe)	3
1.1.2	Information Leakage (Fuga di Informazioni)	4
1.1.3	Gestione Locale vs Globale	4

2	Tabella dati e CWE mappate	4
2.1	CWE	5
2.1.1	CWE-209 Generation of Error Message Con- taining Sensitive Information	6
2.1.2	CWE-234 Failure to Handle Missing Parameter	8
2.1.3	CWE-248 Uncaught Exception	9
2.1.4	CWE-252 Unchecked Return Value	11
2.1.5	Sintesi delle restanti CWE e categorizzazione dei pattern	12
3	Esperimenti svolti e Dimostrazioni pratiche	13
3.1	Strumenti utilizzati	13
3.2	Scenario 1: Iniezione di Input ed Information Leakage (Calcolo Quota)	14
3.2.1	Descrizione dell'esperimento	14
3.2.2	Vulnerabilità ed esecuzione dell'attacco	15
3.2.3	Implementazione della mitigazione	17
3.2.4	Conclusioni raggiunte	19
3.3	Scenario 2: Fallimento non Sicuro e Bypass dell'Autenticazione (Fail-Open)	20
3.3.1	Descrizione dell'esperimento	20
3.3.2	Vulnerabilità ed esecuzione dell'attacco	20
3.3.3	Implementazione della mitigazione	22
3.3.4	Conclusioni raggiunte	24
4	Conclusioni e Valutazioni Personali	24
4.1	Raggiungimento degli Obiettivi	24
4.2	Discussione e Valutazione Personale	25

1 Descrizione

La **Mishandling of Exceptional Conditions** nel software si verifica quando i programmi non riescono a prevenire, rilevare e rispondere a situazioni insolite e imprevedibili, portando a crash, comportamenti inaspettati e, talvolta, vulnerabilità. Ciò può comportare uno o più dei seguenti tre fallimenti: l'applicazione non previene il verificarsi di una situazione insolita, non identifica la situazione mentre sta accadendo e/o risponde male (o non risponde affatto) alla situazione successiva. Le condizioni eccezionali possono essere causate da una convalida dell'input mancante, scarsa o incompleta, oppure da una gestione degli errori tardiva e ad alto livello invece che all'interno delle funzioni in cui si verificano. Altre cause includono stati ambientali imprevisi — come problemi di memoria, di privilegi o di rete — una gestione incoerente delle eccezioni o eccezioni del tutto non gestite, che permettono al sistema di cadere in uno stato ignoto e imprevedibile. Ogni volta che un'applicazione è incerta sulla sua istruzione successiva, significa che una condizione eccezionale è stata gestita male. Errori ed eccezioni difficili da individuare possono minacciare la sicurezza dell'intera applicazione per lungo tempo. Molte diverse vulnerabilità di sicurezza possono manifestarsi quando si gestiscono male le condizioni eccezionali, come bug logici, overflow, race condition, transazioni fraudolente o problemi relativi alla memoria, allo stato, alle risorse, alla tempistica, all'autenticazione e all'autorizzazione. Questi tipi di vulnerabilità possono influenzare negativamente la riservatezza, la disponibilità e/o l'integrità di un sistema o dei suoi dati. Gli aggressori manipolano la gestione degli errori difettosa di un'applicazione per colpire queste vulnerabilità.

1.1 Perché Mishandling of Exceptional Condition è Critica

Per comprendere appieno il rischio, è utile analizzare come un errore apparentemente innocuo possa trasformarsi in una falla di sicurezza.

1.1.1 Il Principio del "Fallimento Sicuro" (Fail-Safe)

Uno dei problemi principali è la risposta inadeguata. In ingegneria del software, un sistema dovrebbe sempre fallire in modo sicuro.

Ad esempio: se un sistema di autenticazione incontra un errore imprevisto (es. il database non risponde), la gestione dell'eccezione deve negare l'accesso di default. Se l'eccezione non è gestita bene, il programma potrebbe "saltare" il controllo e concedere l'accesso per errore.

1.1.2 Information Leakage (Fuga di Informazioni)

Spesso, quando un'eccezione non viene catturata correttamente, il sistema restituisce all'utente un messaggio di errore dettagliato (stack trace). Queste informazioni sono oro colato per un attaccante, poiché rivelano la struttura del database, le versioni delle librerie utilizzate e i percorsi dei file sul server, facilitando attacchi mirati.

1.1.3 Gestione Locale vs Globale

- Gestione Locale: l'errore viene catturato dove nasce. Il programma può tentare di ripristinare l'operazione o pulire le risorse (chiudere file, rilasciare memoria);
- Un "catch-all" alla fine del programma spesso si limita a registrare l'errore e chiudere l'app. Questo lascia spesso "pendenti" connessioni aperte o file corrotti, creando instabilità.

2 Tabella dati e CWE mappate

I dati forniti da OWASP per questa categoria sono chiari e precisi. Si può notare come questi errori nella scrittura di codice da parte dei programmatori incidano tanto sulla sicurezza. Di seguito una tabella dei dati forniti.

CWE coinvolte	Tasso di incidenza max	Tasso di incidenza medio	Massima copertura	Copertura media	Media sfruttamento exploit	Media impatto	Occorrenze totali	CWE totali
24	20.67%	2.95%	100.00%	37.95%	7.11	3.81	769,581	3416

Table 1: Statistiche aggregate delle vulnerabilità

2.1 CWE

Da come si evince dalla tabella, le CWE mappate sono 24:

- **CWE-209** Generation of Error Message Containing Sensitive Information;
- **CWE-215** Insertion of Sensitive Information Into Debugging Code;
- **CWE-234** Failure to Handle Missing Parameter;
- **CWE-235** Improper Handling of Extra Parameters;
- **CWE-248** Uncaught Exception;
- **CWE-252** Unchecked Return Value;
- **CWE-274** Improper Handling of Insufficient Privileges;
- **CWE-280** Improper Handling of Insufficient Permissions or Privileges;
- **CWE-369** Divide By Zero;
- **CWE-390** Detection of Error Condition Without Action;
- **CWE-391** Unchecked Error Condition;
- **CWE-394** Unexpected Status Code or Return Value;
- **CWE-396** Declaration of Catch for Generic Exception;
- **CWE-397** Declaration of Throws for Generic Exception;
- **CWE-460** Improper Cleanup on Thrown Exception;
- **CWE-476** NULL Pointer Dereference;
- **CWE-478** Missing Default Case in Multiple Condition Expression;
- **CWE-484** Omitted Break Statement in Switch;
- **CWE-550** Server-generated Error Message Containing Sensitive Information;
- **CWE-636** Not Failing Securely ('Failing Open');

- **CWE-703** Improper Check or Handling of Exceptional Conditions;
- **CWE-754** Improper Check for Unusual or Exceptional Conditions;
- **CWE-755** Improper Handling of Exceptional Conditions;
- **CWE-756** Missing Custom Error Page;

Questa prima mappatura delle CWE evidenzia come l'errore umano causi vulnerabilità informatiche significative. Di seguito verranno analizzate le CWE presentate, facendo una descrizione della CWE, gli errori di implementazione comuni e le relative strategie di mitigazione per incrementare la sicurezza del sistema.

2.1.1 CWE-209 Generation of Error Message Containing Sensitive Information

Si verifica quando un'applicazione rivela dati sensibili all'interno dei messaggi di errore visualizzati all'utente o salvati in log accessibili. Sebbene possa sembrare un problema minore rispetto a una SQL Injection, il CWE-209 è spesso il precursore di attacchi più gravi, poiché fornisce all'attaccante una "mappa" dettagliata dell'infrastruttura interna. L'essenza del problema risiede nel fornire troppi dettagli a un utente che non dovrebbe averne diritto. In una fase di sviluppo, i messaggi di errore dettagliati sono fondamentali per il debug, ma in produzione diventano un mezzo importante per un attaccante. Alcuni esempi di informazioni che potrebbero essere esposte sono:

- Dati dell'Infrastruttura: Versioni del server web, del database o del sistema operativo;
- Logica Applicativa: Nomi di funzioni, variabili o percorsi di file locali (es. `/var/www/html/config.php`);
- Credenziali o Token: Stringhe di connessione al database che includono password (spesso visibili negli stack trace);
- Dati Utente: Informazioni personali (PII) che finiscono accidentalmente nel dump dell'errore.

L'impatto sta nel facilitare la Reconnaissance ad un attaccante, che, conoscendo l'esatta versione di un database o la struttura di una query SQL che è fallita, può costruire un exploit mirato invece di procedere per tentativi.

Esempio di codice vulnerabile e mitigazione

```
1 <?php
2 $conn = new mysqli("localhost", "db_user", "password123",
3     "my_database");
4
5 $id = $_GET['id'];
6 $query = "SELECT username FROM users WHERE id = " . $id;
7
8 $result = $conn->query($query);
9
10 if (!$result) {
11     die("Errore nel database: " . $conn->error . " |
12         Query: " . $query);
13 }
14 ?>
```

Listing 1: Esempio di codice PHP non sicuro

```
1 <?php
2 ini_set('display_errors', 0);
3 ini_set('log_errors', 1);
4
5 try {
6     $conn = new mysqli("localhost", "db_user",
7         "password123", "my_database");
8
9     $stmt = $conn->prepare("SELECT username FROM users
10         WHERE id = ?");
11     $stmt->bind_param("i", $_GET['id']);
12     $stmt->execute();
13     $result = $stmt->get_result();
14 } catch (Exception $e) {
15     // Log dell'errore reale in un file protetto sul
16     // server dello sviluppatore
17     error_log("Database Error: " . $e->getMessage());
18
19     // Messaggio generico per l'utente
20     die("Si e' verificato un errore di sistema. Riprova
21         piu' tardi.");
22 }
23 ?>
```

Listing 2: Esempio di codice PHP sicuro

2.1.2 CWE-234 Failure to Handle Missing Parameter

Si verifica quando un'applicazione si aspetta di ricevere un determinato parametro in ingresso (come un dato da un form web, un'intestazione HTTP o un argomento da riga di comando), ma non controlla se quel parametro sia effettivamente presente prima di elaborarlo. Se il parametro manca, il software può andare in crash (Denial of Service), assumere comportamenti imprevedibili o bypassare controlli di sicurezza fondamentali. Questo difetto nasce da un'eccessiva fiducia del programmatore nei confronti del client o del flusso standard dell'applicazione. Si assume che, poiché l'interfaccia utente (il frontend) invia sempre quel dato, il server lo riceverà sempre. Un attaccante, tuttavia, può intercettare la richiesta e modificarla (tramite proxy come Burp Suite o semplicemente modificando l'URL) per rimuovere deliberatamente il parametro. Le conseguenze principali sono il **bypass della sicurezza** nel caso in cui il parametro sia un token di autorizzazione o una flag, oppure il **Null Pointer Exception/Crash**, ovvero il tentare di manipolare o leggere un oggetto nullo, che genera un'eccezione non gestita e manda in blocco un processo.

La CWE-235 è molto simile alla CWE-234: entrambe trattano della gestione di parametri, la differenza sta nel numero di parametri;

- CWE-234: fallimento nella gestione di parametri **mancanti**;
- CWE-235: errore nella gestione di parametri **extra**.

Esempio di codice vulnerabile e mitigazione

```
1 from flask import Flask, request, render_template_string
2 app = Flask(__name__)
3
4 @app.route('/reset-password', methods=['GET'])
5 def reset_password():
6     # ERRORE: Si tenta di accedere direttamente al
7     # parametro senza verificare se esiste
8     # Se 'token' manca, Flask lancia un'eccezione HTTP 400
9     # interna, ma se si usassero dizionari standard
10    dict['token'] lancerebbe un KeyError bloccando
    l'app.
    token_inviato = request.args['token']
    # Immaginiamo una logica debole dove se il token e'
    # vuoto o nullo per errore di logica, il controllo
    # passa (es. confronto tra None == None)
```



```

11     if token_inviato == database.get_temp_token():
12         return "Accesso consentito alla pagina di reset."
13
14     return "Token non valido.", 403

```

Listing 3: Esempio di codice Python non sicuro

```

1  from flask import Flask, request, abort
2  app = Flask(__name__)
3
4  @app.route('/reset-password', methods=['GET'])
5  def reset_password():
6      # Verifica esplicita della presenza del parametro
7      if 'token' not in request.args or not
8          request.args.get('token'):
9          # Rifiuta immediatamente la richiesta se il
10             parametro manca o e' vuoto
11             return "Richiesta non valida: Parametro 'token'
12                 mancante.", 400
13
14     token_inviato = request.args.get('token')
15
16     # Logica di business sicura
17     if esegui_controllo_token_sicuro(token_inviato):
18         return "Accesso consentito alla pagina di reset."
19
20     return "Token non valido.", 403

```

Listing 4: Esempio di codice Python sicuro

2.1.3 CWE-248 Uncaught Exception

Si verifica quando un'applicazione solleva un'eccezione (un errore durante l'esecuzione) che non viene catturata e gestita da un apposito blocco di codice (come un try-catch). Quando un'eccezione "rimbalza" fino al livello più alto del programma senza trovare un gestore, il runtime del linguaggio (Java, .NET, Python, Node.js, ecc.) è costretto a prendere provvedimenti drastici: solitamente interrompe l'esecuzione del thread o dell'intera applicazione, oppure mostra un dump dell'errore all'utente. Il problema di sicurezza non è la nascita dell'eccezione, ma l'assenza di un meccanismo di "cattura" per gestirla al meglio. Le conseguenze principali sono il **Denial of Service**, ovvero che se l'eccezione non gestita fa crashare l'intero processo del server, l'applicazione diventa istantaneamente indisponibile per tutti gli utenti, **rilascio di Informazioni Sensibili**, come nella CWE 209, e lo **stato inconsistente**, ovvero che se

l'applicazione si interrompe bruscamente a metà di un'operazione critica, i dati nel database rimangono corrotti o inconsistenti.

Esempio di codice e mitigazione

```
1  import org.springframework.web.bind.annotation.*;
2
3  @RestController
4  public class ProductController {
5
6      @GetMapping("/calculate-discount")
7      public String calculateDiscount(@RequestParam int
8          price, @RequestParam int discountFactor) {
9          // ERRORE: Se discountFactor 0, viene lanciata
10             una ArithmeticException.
11             // Non essendoci un blocco try-catch, l'eccezione
12             non viene catturata.
13             int finalPrice = price / discountFactor;
14
15             return "Prezzo finale: " + finalPrice;
16         }
17     }
18 }
```

Listing 5: Esempio di codice Java non sicuro

```
1  import org.springframework.web.bind.annotation.*;
2  import org.springframework.http.ResponseEntity;
3  import org.slf4j.Logger;
4  import org.slf4j.LoggerFactory;
5
6  @RestController
7  public class ProductController {
8      private static final Logger logger =
9          LoggerFactory.getLogger(ProductController.class);
10
11      @GetMapping("/calculate-discount")
12      public ResponseEntity<String>
13          calculateDiscount(@RequestParam int price,
14              @RequestParam int discountFactor) {
15          try {
16              // Il codice potenzialmente pericoloso viene
17              isolato
18              int finalPrice = price / discountFactor;
19              return ResponseEntity.ok("Prezzo finale: " +
20                  finalPrice);
21          } catch (ArithmeticException e) {
22              // Catturiamo l'eccezione specifica e facciamo
23              il log interno per gli sviluppatori
24              logger.error("Errore di calcolo aritmetico:
25                  divisione per zero.", e);
26          }
27      }
28  }
```

```

20
21         //Rispondiamo all'utente in modo sicuro e
           controllato
22     return
           ResponseEntity.badRequest().body("Parametri
           di calcolo non validi.");
23     }
24 }
25 }

```

Listing 6: Esempio di codice Java sicuro

2.1.4 CWE-252 Unchecked Return Value

Si verifica quando un'applicazione esegue una funzione o chiama un metodo che restituisce un valore (spesso indicante il successo o il fallimento dell'operazione), ma ignora completamente quel valore di ritorno procedendo come se tutto fosse andato a buon fine. Questo difetto è particolarmente diffuso in linguaggi a basso livello come il C/C++, ma si manifesta in qualsiasi linguaggio in cui il controllo del risultato di una funzione è lasciato alla discrezione del programmatore. Molte funzioni di sistema, API o metodi interni comunicano l'esito della loro esecuzione tramite un valore di ritorno (es. 0 per successo, -1 per errore, oppure un puntatore NULL se l'allocazione di memoria fallisce). Se il codice non controlla questo valore, l'applicazione continuerà a eseguire le istruzioni successive basandosi su presupposti falsi. Le conseguenze principali sono il **Bypass dei controlli di sicurezza**, ovvero se una funzione che verifica i permessi di un utente o convalida un certificato SSL fallisce e restituisce un codice di errore, ma tale codice viene ignorato, l'applicazione potrebbe concedere l'accesso ai dati protetti, **Crash e Corruzione della Memoria**, ovvero nel momento in cui fallisce l'allocazione di memoria (in C/C++ sarebbe la malloc) e restituisce NULL, non controllare questo valore prima di scrivere in memoria causerà un crash immediato (Null Pointer Dereference) o, peggio, vulnerabilità sfruttabili per l'esecuzione di codice remoto.

```

1  #include <unistd.h>
2  #include <stdio.h>
3
4  void execute_user_task() {
5      // ERRORE: setuid() restituisce 0 in caso di successo
           e -1 in caso di errore. Qui il valore di ritorno
           viene completamente ignorato.

```

```

6     setuid(1001);
7
8     // Se setuid() fallito (es. per mancanza di risorse
9     // o restrizioni del sistema), il programma ancora
10    // in esecuzione come ROOT.
11    printf("Esecuzione del task utente...\n");
12
13    // Questa funzione eseguir azioni critiche pensando
14    // di essere un utente normale, ma l'attaccante
15    // potr sfruttarla con i massimi privilegi di
16    // sistema.
17    do_critical_operation();
18 }

```

Listing 7: Esempio di codice in C non sicuro

```

1  #include <unistd.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  void execute_user_task() {
6      // Controlliamo esplicitamente il valore di ritorno di
7      // setuid()
8      if (setuid(1001) != 0) {
9          // Se l'operazione fallisce, stampiamo l'errore
10         // nel log e interrompiamo l'esecuzione
11         perror("Errore critico: Impossibile rinunciare ai
12         // privilegi di root");
13         exit(EXIT_FAILURE);
14     }
15
16     // Ora siamo certi al 100% che l'applicazione sta
17     // girando con privilegi ridotti
18     printf("Esecuzione del task utente in sicurezza...\n");
19     do_critical_operation();
20 }

```

Listing 8: Esempio di codice in C sicuro

2.1.5 Sintesi delle restanti CWE e categorizzazione dei pattern

Le restanti CWE mappate da OWASP per questa categoria si sviluppano attorno ai medesimi pattern strutturali finora analizzati. Esse possono essere idealmente raggruppate in tre macro-comportamenti fallimentari del software:

1. **Mancata rilevazione dell'anomalia (Unchecked Conditions):** Il programma prosegue l'esecuzione ignorando che uno

stato interno (es. un puntatore nullo o una divisione per zero) è compromesso.

2. **Rilevazione senza mitigazione (Empty Catch Blocks):** L'eccezione viene intercettata dal codice, ma il blocco di gestione rimane vuoto o si limita a un log passivo, senza riportare l'applicazione in uno stato sicuro.
3. **Bypass della logica di controllo (Fail-Open):** Il fallimento di una routine critica (come un controllo di sicurezza o una transazione) produce un'uscita anticipata che, non essendo gestita esplicitamente, concede l'accesso o convalida l'operazione di default.

Compresa la radice teorica comune di queste vulnerabilità, l'analisi si sposterà ora sulla dimostrazione pratica di come tali difetti implementativi possano essere concretamente sfruttati da un attaccante e, successivamente, mitigati in sicurezza.

3 Esperimenti svolti e Dimostrazioni pratiche

In questa sezione si svolgeranno degli esperimenti per dimostrare quanto critica può essere la **Mishandling of Exceptional Conditions**. In particolare, si presenteranno due esperimenti per due scenari differenti. Essi saranno presentati nel seguente modo:

- Descrizione dell'esperimento;
- Presentazione grafica con schermate e spiegazione di cosa sta avvenendo;
- Presentazione grafica con schermate di come è effettuata la mitigazione della vulnerabilità;
- Conclusioni raggiunte;

3.1 Strumenti utilizzati

Prima di passare agli esperimenti, si preferisce chiarire tutti gli strumenti utilizzati, al fine di rendere più chiaro possibile come sono stati svolti gli esperimenti.

- **Docker:** piattaforma open source che permette di creare, distribuire ed eseguire applicazioni all'interno di ambienti isolati chiamati container. In questo caso, Docker è servito per simulare due server di produzione isolati;
- **Docker-compose:** strumento che permette di definire ed eseguire applicazioni multi-container tramite un singolo file di configurazione in formato YAML;
- **Python 3.10:** noto linguaggio di programmazione, utilizzato per la creazione dei siti web (in locale, come vedremo negli esperimenti) e dei software;
- **Flask:** popolare micro-framework open source per lo sviluppo web, scritto in Python, utilizzato per far funzionare un'applicazione web (gestione delle URL e richieste);
- **HTML (HyperText Markup Language):** linguaggio standard utilizzato per creare e strutturare pagine e siti web; esso permette di dire al browser (come Chrome, Safari o Firefox) come disporre i contenuti.

3.2 Scenario 1: Iniezione di Input ed Information Leakage (Calcolo Quota)

Questo esperimento si concentra su un'applicazione aziendale per la ripartizione delle spese di spedizione (su una fattura fissa di 1000.00 EUR), evidenziando come una mancata convalida dell'input accoppiata a una cattiva gestione delle eccezioni possa esporre dati interni sensibili (*Information Leakage*). La **Mishandling of Exceptional Conditions** in questo caso è mappata come **CWE-209**, **CWE-215**, **CWE-369**, **CWE-550**, **CWE-636** e **CWE-756**.

3.2.1 Descrizione dell'esperimento

L'obiettivo dell'attacco è forzare il software a sollevare un'eccezione a basso livello che il sistema non è in grado di intercettare o mascherare. L'applicazione riceve il numero di partecipanti tramite un parametro GET nell'URL (`?partecipanti=`). Un attaccante proverà a inserire deliberatamente:

- Il valore **0**, per causare un'eccezione di divisione per zero (`ZeroDivisionError`);

- Una stringa alfabetica (es. **"payload"**), per mandare in corto circuito il casting matematico della variabile (`ValueError`).

Se il sistema fallisce in modo non sicuro (*Fail-Open* informativo), lo Stack Trace del framework verrà mostrato direttamente nel browser dell'utente, rivelando informazioni sulla struttura del server.

3.2.2 Vulnerabilità ed esecuzione dell'attacco

Il codice sorgente dell'applicazione vulnerabile non effettua alcun controllo preventivo sull'input ed esegue il software con la flag `debug=True` attiva.

```
1  from flask import Flask, request
2
3  app = Flask(__name__)
4
5  @app.route('/calcola-quota', methods=['GET'])
6  def calcola_quota():
7      totale_fattura = 1000.0
8      partecipanti = request.args.get('partecipanti')
9
10     #Nessun controllo sul tipo di input o sullo zero.
11     # L'esecuzione di operazioni aritmetiche su input non
12     # validi scatena un'eccezione a basso livello.
13     quota = totale_fattura / float(partecipanti)
14
15     @app.route('/')
16
17     if __name__ == '__main__':
18         app.run(host='0.0.0.0', port=5000, debug=True)
19         #debug=True scatena l'interfaccia interattiva
```

Listing 9: Codice sorgente dell'applicazione vulnerabile (app-vulnerable.py)

N.B.: non è stato riportato il codice HTML perché risulta ridondante per l'analisi.

Quando l'applicazione viene aperta (nel nostro caso, su Docker nella porta 5001), un utente legittimo naviga sulla home page incontrando un'interfaccia pulita.

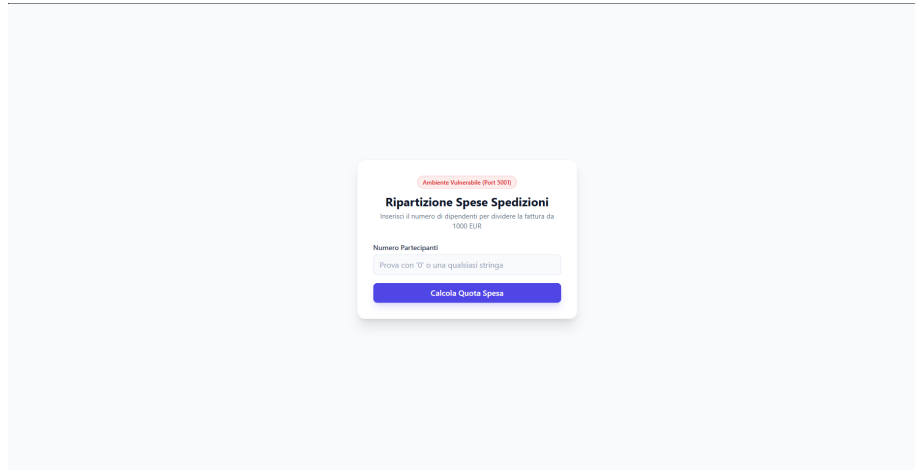


Figure 1: Interfaccia grafica iniziale del calcolatore sul server vulnerabile.

L'attaccante invia quindi la richiesta malevola digitando una **qualsiasi stringa** o il numero 0 all'interno del form. Il backend tenta di convertire la stringa in un numero `float`, fallisce e solleva un'eccezione non gestita. Poiché la modalità di debug è attiva, l'applicazione espone lo Stack Trace interattivo a video.



Figure 2: Stack Trace esposto a video: sono visibili i percorsi dei file e le righe di codice interne.

Analisi del Leak: Come si evince dalla schermata, l'attaccante ha ottenuto informazioni cruciali per la fase di *Reconnaissance*: i percorsi assoluti dei file nel server (`/app/app_vulnerable.py`), la versione esatta del linguaggio e del framework, e le variabili locali in memoria.

3.2.3 Implementazione della mitigazione

La mitigazione del problema applica il principio del **Fail-Safe** agendo su due fronti:

- **Validazione preventiva dell'input:** Si verifica che il parametro non sia vuoto, non sia zero e sia un numero valido prima di effettuare l'operazione aritmetica.
- **Gestore Globale delle Eccezioni (@app.errorhandler):** Qualsiasi errore imprevisto viene intercettato da un Middleware centrale. Lo Stack Trace viene salvato nei log interni del server, mentre all'utente viene mostrata una pagina di errore generica e anonima.

L'applicazione protetta viene eseguita sulla porta 5002 con la modalità di debug rigorosamente disattivata (`debug=False`).

```

1 from flask import Flask, request
2 import logging

```

```

3
4 app = Flask(__name__)
5
6 # Configurazione del logging centralizzato per gli
7   amministratori
8 logging.basicConfig(level=logging.INFO,
9   format='%(asctime)s - %(levelname)s - %(message)s')
10
11 @app.route('/calcola-quota', methods=['GET'])
12 def calcola_quota():
13     totale_fattura = 1000.0
14     partecipanti_raw = request.args.get('partecipanti')
15
16     #Controllo esistenza parametro
17     if not partecipanti_raw:
18         return '<!-- [OMISSIS HTML Grafico] -->
19             <h3>Parametro Mancante</h3>', 400
20
21     try:
22         partecipanti = float(partecipanti_raw)
23         #Controllo divisione per zero (CWE-369)
24         if partecipanti == 0:
25             return '<!-- [OMISSIS HTML Grafico] -->
26                 <h3>Operazione non valida: divisione per
27                 zero</h3>', 400
28
29         quota = totale_fattura / partecipanti
30         return f'<!-- [OMISSIS HTML Grafico] --> <p>Ogni
31             partecipante deve pagare: {quota:.2f} EUR</p>'
32
33     except ValueError:
34         #Input alfabetico malformato
35         return '<!-- [OMISSIS HTML Grafico] --> <h3>Errore
36             Input: formato non valido</h3>', 400
37
38 #Mitigazione Information Leakage
39 @app.errorhandler(Exception)
40 def handle_unexpected_error(error):
41     # Lo Stack Trace reale viene salvato in sicurezza nei
42     log interni sul server
43     app.logger.error(f"Eccezione non gestita intercettata
44         in produzione: {error}", exc_info=True)
45
46     # All'utente esterno viene restituita una schermata
47     anonima e controllata
48     return '<!-- [OMISSIS HTML Grafico] --> <h3>500
49         Internal Server Error</h3>', 500
50
51 def home():
52     return '<!-- [OMISSIS HTML: Form di input grafico per

```

```

42         l\'utente] -->'
43 if __name__ == '__main__':
44     # debug=False
45     app.run(host='0.0.0.0', port=5000, debug=False)

```

Listing 10: Codice sorgente dell'applicazione mitigata (app_secure.py)

Inviando lo stesso input maligno all'applicazione protetta, il sistema intercetta l'anomalia e fallisce in sicurezza. L'utente non vede più alcuna riga di codice, ma un box di avviso controllato.

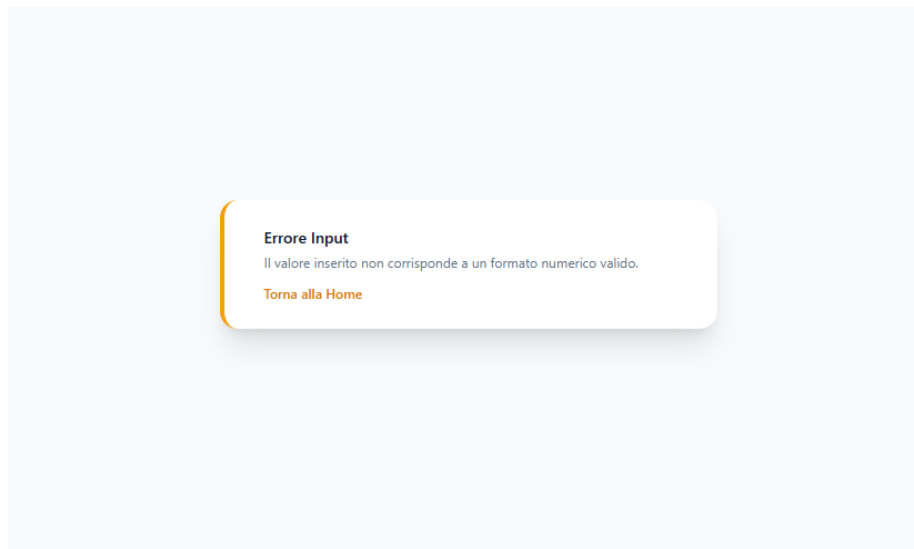


Figure 3: Schermata di errore generica presentata all'utente nel sistema mitigato.

3.2.4 Conclusioni raggiunte

L'esperimento dimostra che la *Mishandling of Exceptional Conditions* non richiede necessariamente tecniche di iniezione complesse per fare danni. Un semplice input malformato può costringere un programma non protetto a rivelare la propria architettura interna. La separazione tra i log di sistema (destinati agli amministratori) e i messaggi di interfaccia (destinati agli utenti) rappresenta una contromisura elementare ma imprescindibile per garantire il principio del minimo privilegio informativo.

3.3 Scenario 2: Fallimento non Sicuro e Bypass dell'Autenticazione (Fail-Open)

Questo secondo esperimento analizza un Gateway di autenticazione a due fattori (2FA) basato su codici temporanei OTP (*One-Time Password*), dimostrando le conseguenze logiche di una gestione delle eccezioni che viola il principio del *Fail-Safe*, permettendo a un attaccante di accedere a un'area riservata senza conoscere le credenziali.

3.3.1 Descrizione dell'esperimento

L'obiettivo dell'attacco è indurre una condizione di errore imprevista nel meccanismo che convalida il codice OTP. L'applicazione si appoggia a un microservizio esterno per verificare la correttezza del codice inserito. Se il servizio esterno è offline, irraggiungibile o sperimenta un timeout, il codice del backend solleva un'eccezione di rete. Un attaccante simulerà questo stato iniettando un input specifico (payload) ideato per forzare il crash del modulo di comunicazione. L'esperimento evidenzierà la differenza di comportamento tra:

- **Il sistema vulnerabile:** che in caso di eccezione del server OTP fallisce in modalità *Fail-Open*, concedendo l'accesso per non interrompere il servizio dell'utente;
- **Il sistema sicuro:** che applica il principio del *Fail-Safe*, negando l'accesso di default in qualunque situazione di incertezza.

3.3.2 Vulnerabilità ed esecuzione dell'attacco

L'applicazione insicura inizializza la variabile di stato dell'autenticazione, ma commette l'errore fatale di alterare tale stato all'interno del blocco di cattura dell'eccezione, interpretando il guasto tecnico come un semaforo verde.

```
1  from flask import Flask, request
2
3  app = Flask(__name__)
4
5  def controlla_otp_esterno(otp):
6      if otp == "payload":
7          raise ConnectionError("Impossibile contattare il server
8              OTP esterno (Timeout).")
9      return otp == "123456"
10
11 @app.route('/login', methods=['POST'])
12 def login():
```

```

12     username = request.form.get('username')
13     otp = request.form.get('otp')
14     autenticato = False
15
16     try:
17         if controlla_otp_esterno(otp):
18             autenticato = True
19     except Exception as e:
20         print(f"[LOG SERVER] Errore critico di rete: {e}")
21         # FALLIMENTO NON SICURO (Fail-Open):
22         # L'eccezione viene catturata, ma il flusso assegna True
23         # alla variabile!
24         autenticato = True
25
26     if autenticato:
27         return '<!-- [OMISSIS HTML Grafico] --> <h1>ACCESSO
28             CONCESSO</h1> FLAG: IS{FAIL_OPEN_SUCCESS}'
29     else:
30         return '<!-- [OMISSIS HTML Grafico] --> <h1>ACCESSO
31             NEGATO</h1>', 401
32
33 @app.route('/')
34 def home():
35     return '<!-- [OMISSIS HTML: Form di Login con Badge Ambiente
36         Vulnerabile] -->'
37
38 if __name__ == '__main__':
39     app.run(host='0.0.0.0', port=5000, debug=False)

```

Listing 11: Codice ottimizzato del login insicuro (auth_vulnerable.py)

L'applicazione viene eseguita tramite l'orchestrazione Docker sulla porta 5003. Inserendo credenziali errate casuali, l'applicazione risponde correttamente negando l'accesso. Tuttavia, inviando la stringa `payload`, il sistema genera l'eccezione programmata, entra nel blocco `except` e bypassa il controllo, mostrando la flag amministrativa.

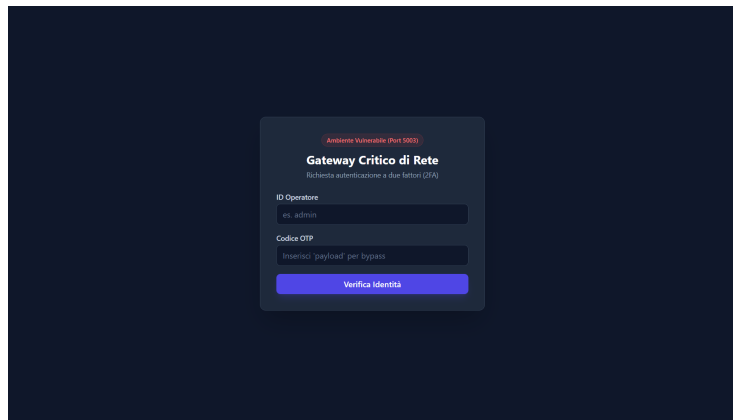


Figure 4: Interfaccia di login sul server vulnerabile (Porta 5003).

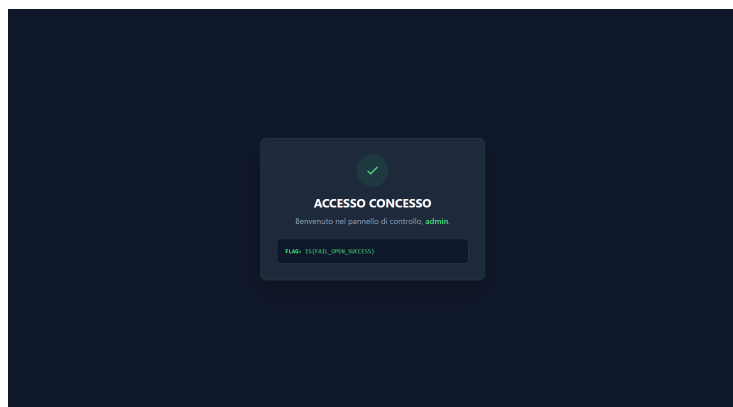


Figure 5: Bypass logico riuscito: schermata di accesso concesso e cattura della flag.

3.3.3 Implementazione della mitigazione

Per mitigare questa vulnerabilità logica, l'applicazione sicura (esposta sulla porta 5004) adotta una politica difensiva rigorosa. Se il blocco `try` non viene completato con successo o se il server OTP esterno lancia un'eccezione di rete, l'applicazione devia immediatamente il flusso verso un'uscita d'emergenza protetta, negando l'accesso all'utente ed emettendo un codice di errore 500 `Internal Server Error`.

```

1  from flask import Flask, request
2
3  app = Flask(__name__)
4
5  def controlla_otp_esterno(otp):
6      if otp == "payload":
7          raise ConnectionError("Impossibile contattare il server
8              OTP esterno (Timeout).")
9      return otp == "123456"
10
11 @app.route('/login', methods=['POST'])
12 def login():
13     username = request.form.get('username')
14     otp = request.form.get('otp')
15
16     try:
17         # L'accesso viene concesso SOLO se la funzione esterna
18         # restituisce esplicitamente True
19         if controlla_otp_esterno(otp):
20             return '<!-- [OMISSIS HTML Grafico] --> <h1>ACCESSO
21                 CONCESSO</h1>'
22         else:
23             return '<!-- [OMISSIS HTML Grafico] --> <h1>ACCESSO
24                 NEGATO</h1>', 401
25
26     except Exception as e:
27         # L'eccezione blocca preventivamente l'autenticazione.
28         print(f"[LOG SERVER SICURO] Eccezione intercettata: {e}")
29         return '<!-- [OMISSIS HTML Grafico] --> <h1>ERRORE DI
30             SISTEMA: Accesso Negato</h1>', 500
31
32 @app.route('/')
33 def home():
34     return '<!-- [OMISSIS HTML: Form di Login con Badge Ambiente
35         Protetto] -->'
36
37 if __name__ == '__main__':
38     app.run(host='0.0.0.0', port=5000, debug=False)

```

Listing 12: Codice ottimizzato del login sicuro (auth_secure.py)

Inviando la stringa *payload* all'indirizzo protetto, il tentativo di bypass logico fallisce. L'eccezione viene intercettata in modo corretto e l'utente viene reindirizzato a una schermata che notifica il malfunzionamento del servizio, senza esporre alcuna risorsa interna o token di sessione.

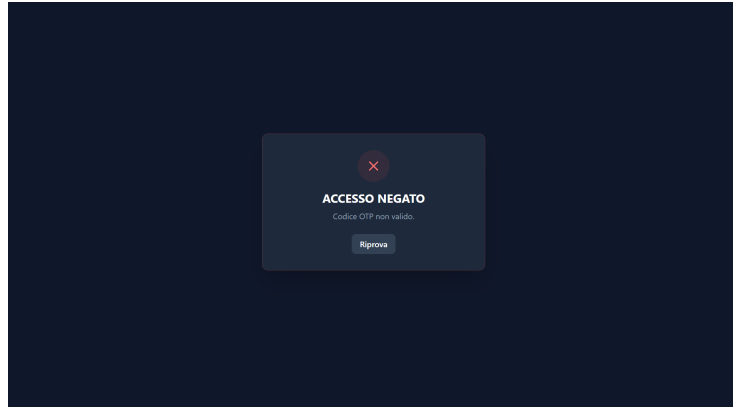


Figure 6: Interfaccia sicura che blocca l'attacco e restituisce una schermata controllata.

3.3.4 Conclusioni raggiunte

Questo esperimento mette in luce l'estrema pericolosità delle falle logiche derivate da scelte implementative che prediligono la disponibilità del servizio (*Availability*) a discapito della sicurezza (*Confidentiality*). In ingegneria della sicurezza del software, la gestione delle eccezioni deve basarsi sul presupposto che qualunque anomalia debba essere trattata come una potenziale minaccia, forzando lo stato dell'applicazione verso il livello di privilegio minimo possibile (ovvero, il rifiuto della richiesta).

4 Conclusioni e Valutazioni Personali

Il presente progetto ha permesso di approfondire in chiave sia teorica che pratica la categoria **OWASP A10:2025 Mishandling of Exceptional Condition**, evidenziando come difetti implementativi apparentemente banali nella gestione delle anomalie possano tramutarsi in falle critiche per la sicurezza di un'intera infrastruttura.

4.1 Raggiungimento degli Obiettivi

Attraverso lo studio dello stato dell'arte e la mappatura delle 24 CWE fornite dal Mitre, si è compreso che il fulcro del problema

risiede quasi sempre nell'assenza di una strategia centralizzata e strutturata per la gestione degli errori.

I due scenari macro progettati e implementati all'interno di questo lavoro hanno dimostrato empiricamente i due principali vettori di rischio:

- **Lo Scenario Finanziario (Porte 5001/5002):** ha evidenziato il pericolo dell'*Information Leakage*. Si è dimostrato come l'esposizione intenzionale o accidentale dello Stack Trace fornisca a un utente malintenzionato una mappa dettagliata del backend (percorsi, versioni delle librerie, logica interna), riducendo drasticamente il tempo necessario per la pianificazione di un attacco mirato.
- **Lo Scenario di Autenticazione (Porte 5003/5004):** ha concretizzato il rischio del *Bypass Logico*. Questo esperimento ha confermato che l'applicazione errata delle priorità nel flusso del software può portare a una configurazione di tipo *Fail-Open*, dove un guasto del sistema si traduce in una concessione indebita di privilegi.

4.2 Discussione e Valutazione Personale

A parere di chi scrive, la categoria *Mishandling of Exceptional Condition* rappresenta una delle sfide più insidiose nella moderna ingegneria del software. Spesso, l'attenzione della sicurezza informatica si concentra su vulnerabilità più "celebri" o esotiche (come *SQL Injection* o *Remote Code Execution*), portando i team di sviluppo a sottovalutare l'importanza della robustezza del codice di fronte a input anomali o stati ambientali imprevisti. Quando un programmatore scrive un blocco di codice, l'attenzione è quasi sempre focalizzata sul flusso in cui tutto funziona correttamente. Tuttavia, dal punto di vista della *Internet Security*, il vero software sicuro si definisce da come si comporta nel momento esatto in cui fallisce. Prediligere la disponibilità del servizio a tutti i costi, lasciando che un'eccezione non gestita porti l'applicazione in uno stato ignoto, è un errore concettuale. Come dimostrato nella mitigazione del sistema OTP, sacrificare momentaneamente l'accessibilità di una risorsa (restituendo un errore 500 controllato) per preservare la riservatezza e l'integrità dei dati è l'unica scelta accettabile in un perimetro di rete critico. In

conclusione, l'adozione di pratiche di *Secure Coding* — quali la validazione rigorosa degli input in ingresso, la disattivazione tassativa delle modalità di debug in produzione e l'incapsulamento delle routine sensibili in blocchi centralizzati di *catch-all* — non costituisce solo un meccanismo di difesa contro gli attaccanti, ma rappresenta il pilastro fondamentale per la stabilità e l'affidabilità del software nella cybersicurezza.